



Ministère de l'Enseignement Supérieur et de la Recherche
Scientifique
Université de Carthage
Institut supérieur des technologies de l'information et de la
communication



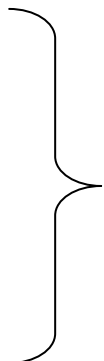
Chapitre 1 : Concepts de base

Plan

1. Structure d'un programme C
2. Les structures de données
3. Opérateurs standards
4. Les instructions
5. Priorités des opérateurs
6. Les conversions forcées de type (casting)

1. Structure d'un programme C

```
#include <stdio.h>
void main()
{
<Déclaration>
< Instructions>
}
```



Chaque programme en C doit renfermer les fichiers include et une fonction main.

Les fichiers include:

- Se trouvent dans un répertoire nommé "include",
- Ils possèdent l'extension h,
- Ils renferment les déclarations des fonctions du système

Exemples :

#include <stdio.h> : fonctions d'entrées/sorties.
#include <math.h> : fonctions mathématiques.
#include <graph.h> : fonctions graphiques.
#include <stdbool.h> : pour utilisation des variables booléens.

La fonction main représente la fonction principale de chaque programme C. Elle comporte deux parties: une partie déclarative (déclaration des variables, constantes, types, ...) et une partie d'exécution des instructions (incrémentations, affectation, appellation des fonctions, des opérations mathématiques, ...)

Remarques :

Un programme en C peut être commenté. Un commentaire est une suite de caractères placés entre des délimiteurs /* */. L'objectif de l'utilisation des commentaires est de rendre le programme plus lisible et compréhensible.

Il existe en C plusieurs couples de symboles qui contrôlent l'affichage ou l'impression du texte. Les séquences d'échappement sont toujours précédées par le caractère d'échappement "\".

\t tabulation

\n nouvelle ligne

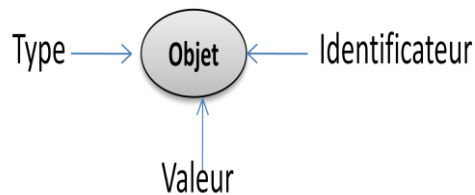
\f saut de page

\r retour chariot (retour au début de ligne, sans saut de ligne)

\a attention (signal acoustique)

2. Les structures de données

Un ordinateur manipule les données d'un tel problème sous forme des objets. Chaque objet a besoin de trois qualificatifs :



- Un objet peut être une variable ou une constante. Une constante est un objet dont la valeur ne varie pas tout au long du programme. Un objet dont la valeur change est nommé variable.

Syntaxe de déclaration d'une variable :

Type Nom_var₁, Nom_var₂, ..., Nom_var_n [= initialisation] ;

Syntaxe de déclaration d'une constante :

const Type Nom_const₁=val₁, Nom_const₂=val₂, ...,
Nom_const_n= val_n ;

- Identificateur: Nom de l'objet

C'est une lettre ou groupe de lettres et des chiffres commençant obligatoirement par une lettre et ne contenant ni espace ni opérateur. Le C distingue les minuscules et les majuscules

Variable	Correcte / Incorrecte
A	Correcte
b	Correcte
abc	Correcte
A17	Correcte
1A	Incorrecte
A+b	Incorrecte
?g	Incorrecte
Ab c	Incorrecte

- Type : Un type de données définit un ensemble dans lequel les variables prennent leur valeur. Chacune des variables dans un programme C doit être associée à un seul type de données. Cinq types de base sont distingués : Entier, Réel, Booléen, Caractère, Chaîne.

2.1. Le type entier (int en langage C)

Une variable de type entier prend ses valeurs de l'ensemble des nombres entiers. Le type int peut être précédé d'un attribut de précision (short ou long) et/ou d'un attribut de représentation (unsigned).

– Les types dérivés du type int

Type	Désignation	Espace	Plage de valeurs
entier court	short int (short)	2 octets	-32768 à 32767
entier court non signé	unsigned short	2 octets	0 à 65535
Entier (type standard)	int	2 octets	-32768 à 32767
entier non signé	unsigned int	2 octets	0 à 65535
entier long	long int (long)	4 octets	-2 147 483 648 à 2 147 483 647
entier long non signé	unsigned long	4 octets	0 à 4 294 967 295

- ✓ Les mots clés short et long permettent d'influer sur la taille mémoire des entiers et des réels.
- ✓ Les deux mots signed et unsigned peuvent s'appliquer aux types caractère et entier pour indiquer si le bit de poids fort doit être considéré ou non comme bit de signe.
- ✓ Par défaut, les types entiers short, int et long sont munis d'un signe. De ce fait, cela revient au même d'écrire : int et signed int, short et signed short, long et signed long.

2.2. Le type réel (float en langage C)

Le type réel est utilisé pour stocker des nombres décimaux (nombres avec une valeur à virgule flottante).

Exemple : Nombre = 14.8

Représentation 1 : $1.48 * 10^1$

Représentation 2 : $0.148 * 10^2$

Représentation 3 : $148.0 * 10^{-1}$

En général, un réel est représenté sous la forme suivante :

<signe><mantisse> *10<exposant>

Avec <signe> : (+) (-)

<mantisse> : décimal positif

<exposant> : entier relatif

– Les types dérivés du type float

Type	Désignation	Nb chiffres mantisse	Espace	Plage de valeurs
réel simple précision	float	6	4 octets	-3.4*10E38 à 3.4*10E38
réel double précision	double	15	8 octets	-1.7*10E308 à 1.7*10E308
réel avec précision étendue	long double	19	10 octets	-3.4*10E4932 à 3.4*10E4932

2.3. Le type caractère (char en langage C)

Ce type représente l'ensemble fini et ordonné de tous les signes qui peuvent être représentés par la machine : lettres minuscules, lettres majuscules, chiffres et signes spéciaux. La plupart du temps, un objet de type char est codé sur un octet ; c'est l'objet le plus élémentaire en C. Le jeu de caractères utilisé correspond généralement au codage ASCII (sur 7 bits).

Exemple :

- Le caractère 'A' est stocké sous forme char
- L'ordinateur n'écrit pas le signe A, mais le nombre 65 (code ASCII de 'A') sur 1 octet = 01000001.

Ils existent 2 types différents :

Type	Désignation	Espace	Plage de valeurs
caractère	char	1 octets	-128 – 127
caractère non signé	unsigned char	1 octets	0 – 255

Une chaîne de caractères est formée par une suite de caractères qui peuvent aller jusqu'à 256 caractères au maximum. En langage C, une chaîne de caractères est un tableau, comportant plusieurs données de type char, dont le dernier élément est le caractère nul '\0'.

Syntaxe de déclaration d'une chaîne

```
char Nom_du_tableau[Nombre_d_elements] ;
```

2.4. Le type booléen (bool en langage C)

Une variable de type booléen doit prendre comme valeur **true** (ou 1) ou **false** (ou 0).

En langage C, pour déclarer des variables de type bool, il est nécessaire d'appeler la bibliothèque associée en utilisant `#include <stdbool.h>`.

3. Opérateurs standards

Les opérateurs arithmétiques : Ils sont utilisés pour effectuer des opérations arithmétiques/mathématiques sur les opérandes.

- **Addition (+):** ajoute deux opérandes.
- **Soustraction (-):** soustrait deux opérandes.
- **Multiplication (*):** multiplie deux opérandes.
- **Division (/):** divise le premier opérande par le second.
- **Module (%):** renvoie le reste lorsque le premier opérande est divisé par le second.

Les opérateurs relationnels : Ils sont utilisés pour comparer deux valeurs. Le résultat de ces opérateurs est une valeur logique **True (1)/False (0)**. **True** lorsque l'opération est vérifiée. **False** lorsque l'opération n'est pas vérifiée.

Opérateur	Symbole en C
Strictement inférieur à	<
Inférieur ou égal à	<=
Strictement supérieur à	>
Supérieur ou égal à	>=
Égal à	=
Différent à	!=

Les opérateurs logiques: Ils sont utilisés pour combiner deux ou plusieurs conditions / contraintes ou pour compléter l'évaluation de la condition d'origine considérée.

ET logique (&& en langage C): retourne **true** lorsque les deux conditions considérées sont vérifiées. Sinon, il retourne **false**.

OU logique (|| en langage C) : renvoie la valeur **true** lorsque l'une (ou les deux) des conditions considérées est vérifiée. Sinon, il retourne **false**.

C1	C2	C1 ET C2	C1 OU C2
F	F	F	F
F	T	F	T
T	F	F	T
T	T	T	T

NON logique (! En langage C) : Renvoie **true** si la condition considérée n'est pas remplie. Sinon, il retourne **false**.

C	NON(C)
F	T
T	F

Exemples:

- $(a < b) \&\& (c < d)$: Prend la valeur 1(vrai) si les deux expressions $a < b$ et $c < d$ sont toutes deux vraies et la valeur 0 dans le cas contraire.

- $(a < b) || (c < d)$: Prend la valeur 1 (vrai) si l'une au moins des deux conditions $a < b$ et $c < d$ est vraie et la valeur 0 dans le cas contraire.
- $!(a < b)$: Prend la valeur 1 (vrai) si la condition $a < b$ est fausse et la valeur 0 (faux) dans le cas contraire.

4. Les instructions

4.1. Affectation

Cette action permet d'affecter une valeur à une variable.

Syntaxe : `Nom_var=val ;`

Val peut être une constante (exemple `Moy= 14 ;`), une variable (exemple `A=B ;`) ou une expression arithmétique (exemple : `Som=X+Y`) ou logique (exemple : `etat=OK1 et OK2 ;`)

Remarque

En langage C les affectations peuvent être représentées de façon combinées, comme suit :

`X+=Y;` est équivalente à `X=X+Y;`

`X-=Y;` est équivalente à `X=X-Y;`

`X*=Y;` est équivalente à `X=X*Y;`

`X/=Y;` est équivalente à `X=X/Y;`

`X%=Y;` est équivalente à `X=X%Y;`

4.2. Les entrées-sorties

La bibliothèque standard `<stdio.h>` contient un ensemble de fonctions qui assurent la communication de la machine avec le monde extérieur. Dans ce chapitre ; nous allons discuter les plus importantes :

- `printf()` écriture formatée de données
- `scanf()` lecture formatée de données
- `putchar()` écriture d'un caractère
- `getchar()` lecture d'un caractère

Écriture formatée des données (`printf` en langage c) : La fonction `printf` est utilisée pour afficher du texte, des valeurs de variables ou des résultats d'expressions à l'écran.

Syntaxe :

`Printf("<format> ", Expr1 , Expr2, ..., ExprN) ;`

- `"<Format> "` : Format de représentation
- `< Expr1, ..., ExprN >` : correspondent aux valeurs des variables ou des expressions spécifiées dans la partie format.
- La partie `"< Format >"` est en fait une chaîne de caractères qui peut contenir : Du texte, des séquences d'échappement et/ou des spécificateurs de format.
 - Les spécificateurs de format indiquent la manière dont les valeurs des expressions `< Expr1 >`, ..., `< ExprN >` sont affichées.
- La partie `"< Format >"` contient exactement un spécificateur de format pour chaque expression `< Expr1 >`, ..., `< ExprN >`.

Exemples:

- `int i = 5; printf ("%d",i);` Résultat : 5
- `printf (" La somme 2+3 donne %d",2+3);` Résultat : La somme 2+3 donne 5
- `int i = 5; printf ("%d plus %d donne : %d", 1000, i, i+1000);` Résultat : 1000 plus 5 donne : 1005.

Lecture formatée des données (scanf en langage c) : La fonction "scanf" est la fonction symétrique à printf. Elle permet d'attribuer une valeur à une variable. Elle nous offre pratiquement les mêmes conversions que printf, mais en sens inverse.

Syntaxe :

`scanf ("", &Var1, &Var2, ... , &VarN);`

- "< Format >" : Format de lecture des données.
- < &V ar1 >, ... : adresses des variables auxquelles les données seront attribuées.
- La chaîne de format détermine comment les données reçues correctement sont mémorisées successivement aux adresses indiquées par < &V ar1 >, < &V ar2 >,
- L'opérateur d'adressage & applique au nom de la variable, il informe donc la fonction "scanf" de l'emplacement de stockage de la valeur saisie.

Exemples:

`int jour, mois, annee ;`

`float x ;`

`scanf ("%d%d%d",&jour,&mois,&annee);`

`scanf ("%f",&x);`

Les macro getchar() et putchar()

getchar() : permet de lire un caractère isolé depuis le clavier et le met à la disposition du programme.

putchar() : affiche un caractère non formaté sur l'écran. La donnée à afficher est écrite sous forme de paramètre entre les parenthèses de la macro.

Exemple

`#include<stdio.h>`

`void main()`

`{`

`char carlu;`

`printf("Entrez une lettre \n");`

`carlu = getchar();`

`putchar (carlu);`

`}`

4.3. Incrémentation et décrémentation

Les opérateurs d'incrémentation et de décrémentation unaires ++ et -- augmentent (incrémentent) ou diminuent (décrémentent) la valeur d'une variable de la quantité 1.

- `x++`; est équivalente à `++x`; est équivalente à `x=x+1`;
- `x--`; est équivalente à `--x`; est équivalente à `x=x-1`;

Post-fixe :

Un opérateur de post-incrémentation ou de post-décrémentation entraîne que son opérande est d'abord utilisé, puis incrémenté ou décrémenté.

Exemple

// Exemple de post-incrémentation

```
int x=1,y;
```

```
y=x++;
```

Résultat : La variable y reçoit la valeur de la variable x, avant que celle-ci ne soit incrémentée, donc y=1, puis x augmente de 1 (x=2).

Pré-fixe :

Un opérateur de pré-incrémentation ou de pré-décrémentation entraîne que son opérande est incrémenté ou décrémenté avant d'être utilisé.

Exemple

// Exemple de post-incrémentation

```
int x=1,y;
```

```
y=++x;
```

Résultat : x augmente de 1 (x=2) et y ne prend pas la valeur d'origine de x, mais la valeur accrue de 1 (y= 2).

4.4.Les fonctions arithmétiques standards

Les fonctions suivantes sont prédéfinies dans la bibliothèque standard. Pour pouvoir les utiliser, le programme doit contenir la ligne : `#include <math.h>`

Type des données : Les arguments et les résultats des fonctions arithmétiques sont de type double.

Nom	Explication
log(X)	logarithme naturel
log10(X)	logarithme à base 10
exp(X)	fonction exponentielle
pow(X,Y)	X exposant Y
sqrt(X)	racine carrée de X
fabs(X)	valeur absolue de X
floor(X)	arrondir en moins
ceil(X)	arrondir en plus
fmod(X,Y)	reste rationnel de X/Y pour X différent de 0
sin(X) cos(X) tan(X)	sinus, cosinus, tangente de X

5. Priorités des opérateurs

Priorité	Opérateur	Ordre d'évaluation
1 (la plus forte)	()	de gauche à droite
2	!, ++, --	de droite à gauche
3	*, /, %	de gauche à droite
4	+, -	de gauche à droite
5	<, <=, >, >=	de gauche à droite
6	==, !=	de gauche à droite
7	&&	de gauche à droite
8		de gauche à droite
9 (la plus faible)	=, + =, - =, * =, / =, % =	de droite à gauche

6. Les conversions forcées de type (casting)

Il est possible de convertir explicitement une valeur en un type quelconque en forçant la transformation à l'aide de la syntaxe :

```
(<type>) <expression >
```

Exemple : Nous divisons deux variables du type entier. Pour avoir plus de précision, nous voulons avoir un résultat de type rationnel. Pour ce faire, nous convertissons l'une des deux opérandes en float. Automatiquement C convertira l'autre opérande en float et effectuera une division rationnelle : `int A=3;`

```
int B=4;
```

```
float C;
```

```
C = (float)A/B;
```

La valeur de A est explicitement convertie en float. La valeur de B est automatiquement convertie en float. Le résultat de la division (type rationnel, valeur 0.75) est affecté à C. Résultat : C=0.75

Attention : Les contenus de A et de B restent inchangés ; seulement les valeurs utilisées dans les calculs sont converties !

Exercice 1

Écrire un programme qui permet de saisir trois réels quelconques, de calculer ensuite leur somme, leur produit et leur moyenne et d'afficher les résultats obtenus.

Solution 1

Algorithme calculs_V1

Variables

nb1, nb2, nb3, Somme, Produit, Moyenne: réel

Début

```
/* saisie des 3 nombres */
écrire ('entrez les trois nombres')
lire (nb1, nb2, nb3)

/* calcul des différents résultats */
Somme ← nb1 + nb2 + nb3
Produit ← nb1 * nb2 * nb3
Moyenne ← somme / 3
/* Affichage des résultats */
écrire ('la somme des trois nombres est : ', Somme)
écrire ('le produit des trois nombres est : ', Produit)
écrire ('la moyenne des trois nombres est : ', Moyenne)
```

Fin

Solution 2

Algorithme calculs_v2

Variables

nb1, nb2, nb3 : réel

Début

```
/* saisie des trois nombres */
écrire ('entrez les trois nombres')
lire (nb1, nb2, nb3)

/* calcul et affichage des différents résultats */
écrire ('la somme des 3 nombres est :', nb1+nb2+nb3)
écrire ('le produit des 3 nombres est :', nb1*nb2*nb3)
écrire ('la moyenne des 3 nombres est :', (nb1+nb2+nb3)/3)
```

Fin

Traduction en C

```
#include<stdio.h>
```

```
void main()
{
float nb1, nb2, nb3, somme, prod, moy;
printf("donner les trois moyennes");
scanf("%f %f %f", &nb1, &nb2,&nb3);
somme=nb1+nb2+nb3;
prod=nb1*nb2*nb3;
moy=somme/3;
printf("somme est %f\n", somme);
printf("produit est %f\n", prod);
printf("moyenne est %f", moy);
}
```

Exercice 2

Ecrire un programme qui permet de déterminer l'air d'un rectangle. Longueur et largeur sont fournies par l'utilisateur.

Algorithme air_rectangle

Variables

long, larg : entier

air :double

Début

Ecrire (" donner le longueur")

Lire(long)

Ecrire (" donner le largeur")

Lire(larg)

Air ← long*larg

Ecrire (" l'air est :", air)

Fin algorithme

Traduction en C

```
#include<stdio.h
```

```
void main()
```

```
{
```

```
float larg , longu;
```

```
double air;
```

```
printf("Entrer la Largeur du rectangle : ");
```

```
scanf("%f",& larg);
```

```
printf("Entrer la Longueur du rectangle : ");
```

```
scanf("%f",&longu);
```

```
air = larg* longu;
```

```
printf("air du rectangle est : %f", air);
```

```
}
```

